DAA IIT2024014

```
 #include <bits/stdc++.h>
using namespace std;

using Matrix = vector<vector<int>>;

Matrix add(const Matrix& A, const Matrix& B) {
    int n = A.size();
    Matrix C(n, vector<int>(n));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            C[i][j] = A[i][j] + B[i][j];
    return C;
}

Matrix sub(const Matrix& A, const Matrix& B) {
    int n = A.size();
    Matrix C(n, vector<int>(n));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            C[i][j] = A[i][j] - B[i][j];
    return C;
}

Matrix strassen(const Matrix& A, const Matrix& B) {
    int n = A.size();
    Matrix C(n, vector<int>(n));
    if (n == 1) {
        C[0][0] = A[0][0] * B[0][0];
        return C;
    }

    int k = n / 2;
    Matrix A11(k, vector<int>(k)), A12(k, vector<int>(k)),
           A21(k, vector<int>(k)), A22(k, vector<int>(k)),
           B11(k, vector<int>(k)), B12(k, vector<int>(k)),
           B21(k, vector<int>(k)), B22(k, vector<int>(k));

    for (int i = 0; i < k; i++)
        for (int j = 0; j < k; j++) {
            A11[i][j] = A[i][j];
            A12[i][j] = A[i][j + k];
            A21[i][j] = A[i + k][j];
```

```
            A22[i][j] = A[i + k][j + k];
            B11[i][j] = B[i][j];
            B12[i][j] = B[i][j + k];
            B21[i][j] = B[i + k][j];
            B22[i][j] = B[i + k][j + k];
        }

    Matrix M1 = strassen(add(A11, A22), add(B11, B22));
    Matrix M2 = strassen(add(A21, A22), B11);
    Matrix M3 = strassen(A11, sub(B12, B22));
    Matrix M4 = strassen(A22, sub(B21, B11));
    Matrix M5 = strassen(add(A11, A12), B22);
    Matrix M6 = strassen(sub(A21, A11), add(B11, B12));
    Matrix M7 = strassen(sub(A12, A22), add(B21, B22));

    Matrix C11 = add(sub(add(M1, M4), M5), M7);
    Matrix C12 = add(M3, M5);
    Matrix C21 = add(M2, M4);
    Matrix C22 = add(sub(add(M1, M3), M2), M6);

    for (int i = 0; i < k; i++)
        for (int j = 0; j < k; j++) {
            C[i][j] = C11[i][j];
            C[i][j + k] = C12[i][j];
            C[i + k][j] = C21[i][j];
            C[i + k][j + k] = C22[i][j];
        }

    return C;
}

int main() {
    srand(time(nullptr));
    int n = 1 << (rand() % 4 + 1);
    Matrix A(n, vector<int>(n)), B(n, vector<int>(n));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            A[i][j] = rand() % 10;
            B[i][j] = rand() % 10;
        }

    Matrix C = strassen(A, B);

    for (auto& r : C) {
```

```
        for (int x : r) cout << x << " ";
        cout << endl;
    }
}
```

Strassen Matrix Multiplication

Time Complexity

- $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$

Space Complexity

- $\Theta(n^2)$

MERGE SORT

```
#include <iostream>

#include <ctime>

#include <vector>

using namespace std;


void merge(vector<int>& array, int left, int mid, int right) {

    int n1 = mid - left + 1;

    int n2 = right - mid;


    vector<int> left_part(n1), right_part(n2);


    for (int i = 0; i < n1; i++)
```

```
        left_part[i] = array[left + i];

    for (int i = 0; i < n2; i++)
        right_part[i] = array[mid + 1 + i];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (left_part[i] <= right_part[j])
            array[k++] = left_part[i++];
        else
            array[k++] = right_part[j++];
    }

    while (i < n1)
        array[k++] = left_part[i++];

    while (j < n2)
        array[k++] = right_part[j++];
}

void mergesort(vector<int>& array, int left, int right) {
    if (left >= right)
        return;
```

```
    int mid = left + (right - left) / 2;

    mergesort(array, left, mid);
    mergesort(array, mid + 1, right);

    merge(array, left, mid, right);
}

int main() {
    vector<int> sample_Array = {45, 25, 1, 30, 66};
    int size_arr = sample_Array.size();

    clock_t start = clock();
    mergesort(sample_Array, 0, size_arr - 1);
    clock_t stop = clock();

    long double time = (long double)(stop - start) / CLOCKS_PER_SEC;

    for (int i = 0; i < size_arr; i++)
        cout << sample_Array[i] << " ";

    cout << "\nTime: " << time << " seconds for size " << size_arr << endl;
    return 0;
```

```
}
```

Merge Sort

Time Complexity

- Best: Θ(n log n)
- Average: Θ(n log n)
- Worst: Θ(n log n)

Space Complexity

- Θ(n)

QUICK SORT

```
#include <bits/stdc++.h>

using namespace std;


int partition(vector<int>& a, int l, int r) {
    int pivot = a[r];
    int i = l - 1;
    for (int j = l; j < r; j++) {
        if (a[j] <= pivot) {
            i++;
            swap(a[i], a[j]);
        }
    }
```

```
    swap(a[i + 1], a[r]);
    return i + 1;
}

void quickSort(vector<int>& a, int l, int r) {
    if (l < r) {
        int p = partition(a, l, r);
        quickSort(a, l, p - 1);
        quickSort(a, p + 1, r);
    }
}

int main() {
    srand(time(nullptr));
    int n = rand() % 20 + 1;
    vector<int> a(n);
    for (int i = 0; i < n; i++)
        a[i] = rand() % 100;

    quickSort(a, 0, n - 1);

    for (int x : a)
        cout << x << " ";
    cout << endl;
```

}

Quick Sort

Time Complexity

- Best: $\Theta(n \log n)$
- Average: $\Theta(n \log n)$
- Worst: $\Theta(n^2)$

Space Complexity

- Average: $\Theta(\log n)$
- Worst: $\Theta(n)$